Q4.1: Create Payable Interface and Implement Salary System

QUESTION:

Create a Payable interface with a calculateSalary() method. Make all staff classes (Manager, HouseKeeper, Receptionist) implement this interface. Each staff type should have different salary calculations:

- Manager: £4000 base salary
- HouseKeeper: £2000 + £100 per year of experience
- Receptionist: £2500 + £50 per language spoken

Add a payroll report feature to display all staff salaries.

SOLUTION:

Step 1: Create Payable.java (NEW FILE)

```
package com.mycompany.hotelmanagementsystem;


public interface Payable {
   double calculateSalary();
   String getSalaryBreakdown();
}
```

Step 2: Update Manager.java

CHANGE THIS LINE:

```
public class Manager extends HotelStaff {
```

TO:

```
public class Manager extends HotelStaff implements Payable {
```

ADD THESE METHODS:

```
@Override
public double calculateSalary() {
   return 4000.0;
}


@Override
```

```
public String getSalaryBreakdown() {
   return "Manager Base Salary: £4000.00";
}
```

Step 3: Update HouseKeeper.java

CHANGE THIS LINE:

```
public class HouseKeeper extends HotelStaff {
```

TO:

```
public class HouseKeeper extends HotelStaff implements Payable {
```

ADD THESE METHODS:

```
@Override
public double calculateSalary() {
   return 2000.0 + (yearsOfExperience * 100.0);
}

@Override
public String getSalaryBreakdown() {
   return "Base: £2000.00 + Experience: £" + (yearsOfExperience * 100.0);
}
```

Step 4: Update Receptionist.java (if exists)

CHANGE THIS LINE:

```
public class Receptionist extends HotelStaff {
```

TO:

```
public class Receptionist extends HotelStaff implements Payable {
```

ADD THESE METHODS:

```
@Override
public double calculateSalary() {
   int languageCount = 1;
   if (languagesSpoken != null && !languagesSpoken.trim().isEmpty()) {
```

```
        languageCount = languagesSpoken.split(",").length;
    }
    return 2500.0 + (languageCount * 50.0);
}

@Override
public String getSalaryBreakdown() {
    int languageCount = 1;
    if (languagesSpoken != null && !languagesSpoken.trim().isEmpty()) {
        languageCount = languagesSpoken.split(",").length;
    }
    return "Base: £2500.00 + Languages: £" + (languageCount * 50.0);
}
```

Step 5: Add to WestminsterHotelManager.java

ADD THESE METHODS:

```
public void printPayrollReport() {
    System.out.println("\n=== PAYROLL REPORT ===");
    double totalPayroll = 0.0;

    for (HotelStaff staff : hotelStaffList) {
        if (staff instanceof Payable) {
            Payable payableStaff = (Payable) staff;
            double salary = payableStaff.calculateSalary();
            totalPayroll += salary;

            System.out.println(staff.getName() + " " + staff.getSurname() +
                    " (" + staff.getClass().getSimpleName() +
                    "): £" + String.format("%.2f", salary));
```

```
    }
  }

  System.out.println("TOTAL PAYROLL: £" + String.format("%.2f", totalPayroll));
}
```

ADD TO MENU (in runMenu method):

```
System.out.println("To View Payroll Report, press 6");

case 6:
  this.printPayrollReport();
  break;
```

Q4.2: Create Custom Exception Classes for Validation

QUESTION:

Create custom exceptions InvalidStaffIDException and DuplicateStaffIDException. Use these exceptions in the staff addition process with proper try-catch blocks. Add validation to ensure Staff IDs follow the format AB1234 (2 letters + 4 numbers) and are unique.

SOLUTION:

Step 1: Create InvalidStaffIDException.java (NEW FILE)

```
package com.mycompany.hotelmanagementsystem;

public class InvalidStaffIDException extends Exception {
  public InvalidStaffIDException(String message) {
    super(message);
  }
}
```

Step 2: Create DuplicateStaffIDException.java (NEW FILE)

```
package com.mycompany.hotelmanagementsystem;
```

```
public class DuplicateStaffIDException extends Exception {
  private String duplicateID;

  public DuplicateStaffIDException(String staffID) {
    super("Staff ID '" + staffID + "' already exists");
    this.duplicateID = staffID;
  }

  public String getDuplicateID() {
    return duplicateID;
  }
}
```

Step 3: Add Validation Methods to WestminsterHotelManager.java

ADD THESE METHODS:

```
public void validateStaffID(String staffID) throws InvalidStaffIDException,
DuplicateStaffIDException {
  // Check format (2 letters + 4 numbers)
  if (staffID == null || !staffID.matches("[A-Z]{2}[0-9]{4}")) {
    throw new InvalidStaffIDException("Staff ID must be 2 letters + 4 numbers (e.g.,
AB1234)");
  }

  // Check for duplicates
  for (HotelStaff staff : hotelStaffList) {
    if (staff.getStaffID().equals(staffID)) {
      throw new DuplicateStaffIDException(staffID);
    }
  }
}
```

Step 4: Update addHotelStaff() Method

FIND THIS SECTION:

```
System.out.println("Enter the staff ID");
String staffID = s.nextLine();
```

REPLACE WITH:

```
System.out.println("Enter the staff ID (Format: AB1234)");
String staffID = "";
boolean validID = false;
int attempts = 0;

while (!validID && attempts < 3) {
    try {
        staffID = s.nextLine().toUpperCase().trim();
        validateStaffID(staffID);
        validID = true;
    } catch (InvalidStaffIDException e) {
        System.out.println("Invalid format: " + e.getMessage());
        attempts++;
        if (attempts < 3) System.out.print("Try again: ");
    } catch (DuplicateStaffIDException e) {
        System.out.println("Duplicate ID: " + e.getMessage());
        attempts++;
        if (attempts < 3) System.out.print("Try again: ");
    }
}

if (!validID) {
    System.out.println("Maximum attempts reached. Staff addition cancelled.");
```

```
    return;
}
```

Q4.3: Create Schedulable Interface for Work Schedule Management

QUESTION:

Create a Schedulable interface with methods for managing work schedules. Implement this interface in the Receptionist class to manage shift times (Day, Evening, Night). Add functionality to set schedules and check staff availability.

SOLUTION:

Step 1: Create Schedulable.java (NEW FILE)

```
package com.mycompany.hotelmanagementsystem;


public interface Schedulable {
    void setWorkSchedule(String schedule);
    String getWorkSchedule();
    boolean isAvailable(String time);
}
```

Step 2: Update Receptionist.java

CHANGE CLASS DECLARATION:

```
public class Receptionist extends HotelStaff implements Payable, Schedulable {
```

ADD FIELD:

```
private String workSchedule;
```

UPDATE CONSTRUCTOR:

```
public Receptionist(String name, String surname) {
    super(name, surname);
    this.workSchedule = "Day Shift (9AM-5PM)";
}
```

ADD SCHEDULABLE METHODS:

```
@Override
```

```
public void setWorkSchedule(String schedule) {
   this.workSchedule = schedule;
}

@Override
public String getWorkSchedule() {
   return workSchedule;
}

@Override
public boolean isAvailable(String time) {
   if (workSchedule.contains("Day")) {
      return time.contains("AM") || time.contains("1PM") || time.contains("2PM") ||
           time.contains("3PM") || time.contains("4PM") || time.contains("5PM");
   } else if (workSchedule.contains("Evening")) {
      return time.contains("2PM") || time.contains("3PM") || time.contains("4PM") ||
           time.contains("5PM") || time.contains("6PM") || time.contains("7PM") ||
           time.contains("8PM") || time.contains("9PM") || time.contains("10PM");
   } else if (workSchedule.contains("Night")) {
      return time.contains("10PM") || time.contains("11PM") || time.contains("12AM") ||
           time.contains("1AM") || time.contains("2AM") || time.contains("3AM") ||
           time.contains("4AM") || time.contains("5AM") || time.contains("6AM");
   }
   return false;
}
```

Step 3: Add Schedule Management to WestminsterHotelManager.java

ADD THESE METHODS:

```
public void setStaffSchedule() {
```

```
Scanner s = new Scanner(System.in);
System.out.println("Enter Staff ID:");
String staffID = s.nextLine();

HotelStaff staff = null;
for (HotelStaff member : hotelStaffList) {
   if (member.getStaffID().equals(staffID)) {
      staff = member;
      break;
   }
}

if (staff != null && staff instanceof Schedulable) {
   System.out.println("Schedule options:");
   System.out.println("1. Day Shift (9AM-5PM)");
   System.out.println("2. Evening Shift (2PM-10PM)");
   System.out.println("3. Night Shift (10PM-6AM)");
   System.out.print("Choose (1-3): ");

   int choice = s.nextInt();
   s.nextLine();

   String schedule = "";
   switch (choice) {
      case 1: schedule = "Day Shift (9AM-5PM)"; break;
      case 2: schedule = "Evening Shift (2PM-10PM)"; break;
      case 3: schedule = "Night Shift (10PM-6AM)"; break;
      default: System.out.println("Invalid choice!"); return;
```

```
        }

        ((Schedulable) staff).setWorkSchedule(schedule);
        System.out.println("Schedule updated!");
    } else {
        System.out.println("Staff not found or doesn't support scheduling.");
    }
}

public void checkStaffAvailability() {
    Scanner s = new Scanner(System.in);
    System.out.println("Enter time to check (e.g., 10AM, 3PM): ");
    String time = s.nextLine();

    System.out.println("Available staff at " + time + ":");
    for (HotelStaff staff : hotelStaffList) {
        if (staff instanceof Schedulable) {
            if (((Schedulable) staff).isAvailable(time)) {
                System.out.println("- " + staff.getName() + " " + staff.getSurname());
            }
        }
    }
}
```

ADD TO MENU:

```
System.out.println("To Set staff schedule, press 7");
System.out.println("To Check availability, press 8");

case 7:
```

```
    this.setStaffSchedule();
    break;
case 8:
    this.checkStaffAvailability();
    break;
```

Q4.4: Enhanced Menu System with Submenus

QUESTION:

Create a professional menu system with submenus for different operations (Staff Management, Guest Management, Reports). Add clear navigation, screen clearing, and back options for better user experience.

SOLUTION:

Step 1: Replace runMenu() Method in WestminsterHotelManager.java

```
@Override
public boolean runMenu() {
    boolean exit = false;

    // Clear screen
    for (int i = 0; i < 50; i++) {
        System.out.println();
    }

System.out.println("╔═══════════════════════════════════════════════════════════╗");
    System.out.println("║        HOTEL MANAGEMENT SYSTEM        ║");
System.out.println("╚═══════════════════════════════════════════════════════════╝");
    System.out.println();
    System.out.println("1. Staff Management");
```

```
System.out.println("2. Guest Management");
System.out.println("3. Reports & Statistics");
System.out.println("4. System Settings");
System.out.println("5. Open GUI Interface");
System.out.println("0. Exit");
System.out.println();
System.out.print("Choice (0-5): ");

Scanner s = new Scanner(System.in);

try {
   int choice = s.nextInt();
   s.nextLine();

   switch (choice) {
      case 0:
         exit = confirmExit();
         break;
      case 1:
         this.staffManagementMenu();
         break;
      case 2:
         this.guestManagementMenu();
         break;
      case 3:
         this.reportsMenu();
         break;
      case 4:
```

```
            this.systemSettingsMenu();
            break;
        case 5:
            this.runGUI();
            break;
        default:
            System.out.println("Invalid choice!");
            pauseForUser();
      }
    } catch (Exception e) {
      System.out.println("Invalid input!");
      s.nextLine();
      pauseForUser();
    }

    return exit;
}
```

Step 2: Add Submenu Methods

```
private boolean confirmExit() {
    Scanner s = new Scanner(System.in);
    System.out.print("Exit? (y/n): ");
    return s.nextLine().toLowerCase().startsWith("y");
}

private void pauseForUser() {
    Scanner s = new Scanner(System.in);
    System.out.print("Press Enter...");
    s.nextLine();
```

```
}

public void staffManagementMenu() {
   boolean back = false;
   Scanner s = new Scanner(System.in);

   while (!back) {
      System.out.println("\n=== STAFF MANAGEMENT ===");
      System.out.println("Staff Count: " + hotelStaffList.size());
      System.out.println("1. Add Staff");
      System.out.println("2. View All Staff");
      System.out.println("3. Search Staff");
      System.out.println("4. Remove Staff");
      System.out.println("5. Payroll Report");
      System.out.println("0. Back");
      System.out.print("Choice: ");

      try {
         int choice = s.nextInt();
         s.nextLine();

         switch (choice) {
            case 0: back = true; break;
            case 1: this.addHotelStaff(); pauseForUser(); break;
            case 2: this.printHotelStaffList(); pauseForUser(); break;
            case 3: this.searchStaffConsole(); pauseForUser(); break;
            case 4: this.removeStaffConsole(); pauseForUser(); break;
            case 5: this.printPayrollReport(); pauseForUser(); break;
```

```
            default: System.out.println("Invalid!"); pauseForUser();
        }
      } catch (Exception e) {
        System.out.println("Invalid input!");
        s.nextLine();
        pauseForUser();
      }
    }
}

public void guestManagementMenu() {
    boolean back = false;
    Scanner s = new Scanner(System.in);

    while (!back) {
      System.out.println("\n=== GUEST MANAGEMENT ===");
      System.out.println("Guest Count: " + hotelGuestList.size());
      System.out.println("1. Add Guest");
      System.out.println("2. View All Guests");
      System.out.println("3. Search by Room");
      System.out.println("4. Remove Guest");
      System.out.println("0. Back");
      System.out.print("Choice: ");

      try {
        int choice = s.nextInt();
        s.nextLine();
```

```
            switch (choice) {
                case 0: back = true; break;
                case 1: this.addGuest(); pauseForUser(); break;
                case 2: this.printGuestList(); pauseForUser(); break;
                case 3: this.searchGuestByRoom(); pauseForUser(); break;
                case 4: this.removeGuestConsole(); pauseForUser(); break;
                default: System.out.println("Invalid!"); pauseForUser();
            }
        } catch (Exception e) {
            System.out.println("Invalid input!");
            s.nextLine();
            pauseForUser();
        }
    }
}

public void reportsMenu() {
    boolean back = false;
    Scanner s = new Scanner(System.in);

    while (!back) {
        System.out.println("\n=== REPORTS & STATISTICS ===");
        System.out.println("1. Hotel Overview");
        System.out.println("2. Staff Statistics");
        System.out.println("3. Financial Report");
        System.out.println("4. Payroll Report");
        System.out.println("0. Back");
        System.out.print("Choice: ");
```

```
        try {
            int choice = s.nextInt();
            s.nextLine();

            switch (choice) {
                case 0: back = true; break;
                case 1: this.printHotelOverview(); pauseForUser(); break;
                case 2: this.printStaffStatistics(); pauseForUser(); break;
                case 3: this.printFinancialReport(); pauseForUser(); break;
                case 4: this.printPayrollReport(); pauseForUser(); break;
                default: System.out.println("Invalid!"); pauseForUser();
            }
        } catch (Exception e) {
            System.out.println("Invalid input!");
            s.nextLine();
            pauseForUser();
        }
    }
}

public void systemSettingsMenu() {
    System.out.println("System settings would be here...");
    pauseForUser();
}
```

Q4.5: Add Contact Information to Staff

QUESTION:

Add phone number and email fields to the HotelStaff class. Update the staff addition process to collect this information and modify the staff display to show contact details.

SOLUTION:

Step 1: Update HotelStaff.java

ADD THESE FIELDS:

```
private String phoneNumber;

private String email;
```

ADD THESE METHODS:

```
public void setPhoneNumber(String phoneNumber) {
   this.phoneNumber = phoneNumber;
}

public String getPhoneNumber() {
   return phoneNumber;
}

public void setEmail(String email) {
   this.email = email;
}

public String getEmail() {
   return email;
}
```

Step 2: Update addHotelStaff() Method

ADD AFTER DATE INPUT:

```
System.out.println("Enter phone number:");
String phoneNumber = s.nextLine();

System.out.println("Enter email address:");
```

String email = s.nextLine();

UPDATE EACH CASE TO SET CONTACT INFO:

// In Manager case:

manager.setPhoneNumber(phoneNumber);

manager.setEmail(email);

// In HouseKeeper case:

housekeeper.setPhoneNumber(phoneNumber);

housekeeper.setEmail(email);

// In Receptionist case:

receptionist.setPhoneNumber(phoneNumber);

receptionist.setEmail(email);

Step 3: Update toString() Methods

ADD TO END OF toString() IN EACH STAFF CLASS:

// In Manager, HouseKeeper, Receptionist toString():

return super.toString() + " [existing info] - Phone: " +

(getPhoneNumber() != null ? getPhoneNumber() : "N/A") +

", Email: " + (getEmail() != null ? getEmail() : "N/A");

Q4.6: Add Data Validation Framework

QUESTION:

Create a ValidationUtils class with static methods for validating email addresses, phone numbers, and staff IDs. Integrate this validation into the staff addition process with proper error handling.

SOLUTION:

Step 1: Create ValidationUtils.java (NEW FILE)

package com.mycompany.hotelmanagementsystem;

```
import java.util.regex.Pattern;

public class ValidationUtils {
    private static final Pattern EMAIL_PATTERN = Pattern.compile(
        "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$"
    );

    private static final Pattern PHONE_PATTERN = Pattern.compile(
        "^(\\+44|0)[0-9]{10}$|^[0-9]{11}$"
    );

    private static final Pattern STAFF_ID_PATTERN = Pattern.compile(
        "^[A-Z]{2}[0-9]{4}$"
    );

    public static boolean isValidEmail(String email) {
        return email != null && EMAIL_PATTERN.matcher(email.trim()).matches();
    }

    public static boolean isValidPhoneNumber(String phone) {
        if (phone == null) return false;
        String clean = phone.replaceAll("[\\s()-]", "");
        return PHONE_PATTERN.matcher(clean).matches();
    }

    public static boolean isValidStaffID(String staffID) {
        return staffID != null &&
STAFF_ID_PATTERN.matcher(staffID.trim().toUpperCase()).matches();
    }
```

```
public static String formatPhoneNumber(String phone) {
    if (!isValidPhoneNumber(phone)) return phone;
    String clean = phone.replaceAll("[\\s()-]", "");
    if (clean.startsWith("0")) {
        clean = "+44" + clean.substring(1);
    }
    return clean;
}

public static class ValidationResult {
    private boolean valid;
    private String errorMessage;
    private String formattedValue;

    public ValidationResult(boolean valid, String errorMessage, String formattedValue) {
        this.valid = valid;
        this.errorMessage = errorMessage;
        this.formattedValue = formattedValue;
    }

    public boolean isValid() { return valid; }
    public String getErrorMessage() { return errorMessage; }
    public String getFormattedValue() { return formattedValue; }
}

public static ValidationResult validateEmail(String email) {
    if (email == null || email.trim().isEmpty()) {
```

```
            return new ValidationResult(false, "Email is required", null);
        }

        if (!isValidEmail(email)) {
            return new ValidationResult(false, "Invalid email format", null);
        }

        return new ValidationResult(true, null, email.trim().toLowerCase());
    }

    public static ValidationResult validatePhone(String phone) {
        if (phone == null || phone.trim().isEmpty()) {
            return new ValidationResult(false, "Phone number is required", null);
        }

        if (!isValidPhoneNumber(phone)) {
            return new ValidationResult(false, "Invalid phone format", null);
        }

        return new ValidationResult(true, null, formatPhoneNumber(phone));
    }
}
```

Step 2: Update Contact Collection in addHotelStaff()

REPLACE PHONE AND EMAIL COLLECTION:

```
// Phone validation
String phoneNumber = "";
boolean validPhone = false;
while (!validPhone) {
```

```
    System.out.println("Enter phone number (UK format):");
    String phoneInput = s.nextLine();

    ValidationUtils.ValidationResult phoneResult =
ValidationUtils.validatePhone(phoneInput);
    if (phoneResult.isValid()) {
        phoneNumber = phoneResult.getFormattedValue();
        validPhone = true;
    } else {
        System.out.println("Error: " + phoneResult.getErrorMessage());
    }
}

// Email validation
String email = "";
boolean validEmail = false;
while (!validEmail) {
    System.out.println("Enter email address:");
    String emailInput = s.nextLine();

    ValidationUtils.ValidationResult emailResult = ValidationUtils.validateEmail(emailInput);
    if (emailResult.isValid()) {
        email = emailResult.getFormattedValue();
        validEmail = true;
    } else {
        System.out.println("Error: " + emailResult.getErrorMessage());
    }
}
```